STruCpp

Software Supply Chain Security Report — SBOM & VEX

Product	STruCpp (`strucpp`) — IEC 61131-3 Structured Text → C++ compiler / transpiler · v0.6.0
Report type	Software Bill of Materials (SBOM) & Vulnerability Exploitability eXchange (VEX)
Assessment date	2026-10 · Report version 1.0
Prepared by	Autonomy Logic — Engineering / Product Security
Classification	Confidential
Security contact	Thiago Alves — thiago.alves@autonomylogic.com

Executive Summary

This report documents the third-party software composition and known-vulnerability posture of **STruCpp**. It is aligned with U.S. Executive Order 14028, the NTIA *Minimum Elements for an SBOM*, and the CISA VEX guidance.

> **Headline posture.** STruCpp is a TypeScript/Node.js command-line compiler distributed as a self-contained binary, with an unusually small third-party runtime footprint (a single direct runtime dependency, `chevrotain`). Of 29 unique advisories across the dependency tree, **none require remediation** and **none are exploitable in the shipped compiler**. Every advisory is either in build/test tooling that is not distributed, or in the one transitive runtime library (`lodash-es`) whose vulnerable functions are never called. Both critical-severity advisories are in development tooling only.

Key metrics

Metric	Value
Components inventoried (full transitive graph)	**764**
Raw advisories detected	29 (2 critical · 12 high · 14 moderate · 1 low)
AFFECTED — action required	**0** (none require action)
AFFECTED — mitigated	0 (—)
NOT AFFECTED	**29** (100%)

1. Scope & System Description

STruCpp is a command-line compiler written in TypeScript, executed on Node.js and distributed as a self-contained executable (built with `pkg` for Linux/Windows/macOS). It parses IEC 61131-3 Structured Text and emits C++. The repository also contains a first-party C/C++ runtime library (headers the emitted C++ links against) — Autonomy Logic's own source code, not a third-party dependency, covered by the Runtime Library Exception.

> **Scope note.** This SBOM covers the Node.js/npm dependency graph of the compiler. The bundled Node.js runtime (embedded by `pkg`) and the first-party C/C++ runtime library are Autonomy Logic artifacts, assessed separately from third-party supply chain. A companion VS Code extension in the same repository is a distinct component.

2. Methodology

Advisory data was obtained from the GitHub Advisory Database via `npm audit` (equivalent to GitHub Dependabot alerts). The compiler is bundled from a committed lockfile; development, build, and test dependencies are not part of the distributed binary — only the runtime dependency subtree (`chevrotain` and its transitives) ships. The compiler's input is a Structured Text source file, treated as untrusted. For each relevant package, source code was analyzed to determine whether the vulnerable code path is invoked and whether its input is attacker-controlled.

3. Software Bill of Materials Summary

License	Components
MIT	619
ISC	44
Apache-2.0	28
BSD-2-Clause	22
BSD-3-Clause	14
BlueOak-1.0.0	12
SEE LICENSE IN LICENSE.txt	10
Artistic-2.0	5

License finding: STruCpp itself is licensed GPL-3.0-or-later, with a GCC-style Runtime Library Exception (RLE). The compiler is open-source copyleft (distributing it or a modified version carries GPL-3.0 obligations, including source availability), while the RLE means C++ code produced by the compiler is **not** forced to be GPL — end users can compile and distribute proprietary PLC programs (mirroring GCC's libstdc++ exception). Copyright is held by Autonomy Logic / the OpenPLC Project. This is the product's intended license, surfaced for the acquirer's IP/legal review. The third-party dependency graph is fully permissive (MIT/ISC/Apache/BSD) — no copyleft is introduced through dependencies.

4. Findings Requiring Remediation (Affected)

None.

5. Not Affected — VEX Justifications

VEX justification (CISA)	Count	Representative components	Basis
`component_not_present`	26	tar (1 Critical + high), vitest (1 Critical), minimatch, vite, esbuild, rollup, brace-expansion, js-yaml, picomatch, postcss, flatted, ajv, yaml	Development, build, and test tooling (bundler, test runner, packager). Not part of the runtime dependency subtree; excluded from the distributed binary. Both critical advisories fall here.
`vulnerable_code_not_in_execute_path`	3	lodash-es (1 high + 2 moderate) — via chevrotain	The only third-party runtime library with advisories. The vulnerable functions (_.template code injection; _.unset/_.omit prototype pollution) are never called: STruCpp does not import lodash directly, and chevrotain uses lodash-es only for internal parser data structures — the compiler's Structured Text input never reaches those functions.

> **On critical severity.** Both critical advisories (tar, vitest) are in development/build tooling and are not part of the distributed compiler. Neither is present in the delivered product.

6. Mitigated Findings

Component	Advisories	Existing control

7. Remediation Plan

- **No action required** for exploitable vulnerabilities — no advisory is both present in the distributed compiler and exploitable.
- **Hygiene:** keep the bundled Node.js runtime (via `pkg`) current with Node security releases.
- **Legal:** the first-party GPL-3.0-or-later + Runtime Library Exception license (§ license posture) is material to the acquirer's IP review.

8. Secure Development & Supply-Chain Practices

Practice	Status
Minimal runtime dependency surface (1 direct dependency)	In place
Deterministic builds from a committed lockfile (npm ci)	In place
Development/build tooling excluded from the distributed binary	In place
Automated dependency updates (Dependabot / GitHub Advisory Database)	In place
SBOM generated per release in CycloneDX and SPDX	In place
Keep the bundled Node.js runtime (via pkg) current with Node security releases	Recommended

9. Attached Artifacts

Artifact	Format	Purpose
`sbom/strucpp.cdx.json`	CycloneDX 1.6	Canonical machine-readable SBOM
`sbom/strucpp.spdx.json`	SPDX 2.3 (ISO/IEC 5962)	Procurement / compliance SBOM
`sbom/strucpp.components.csv`	CSV	Human-readable component inventory (764 rows)
`sbom/vulnerabilities.csv`	CSV	Full annotated advisory register (29 rows)

Prepared by Autonomy Logic Engineering. Assessment date 2026-10. Regenerate per release.